

# Roteiro do vídeo — Construindo um Transformer (LLM) do zero

**Notebook:** `Transformer.ipynb` · **Cenas:** 33 · **Duração estimada:** 10:00 (narração a ~150 palavras por minuto)

Cada cena traz a captura de tela do notebook (texto explicativo, nota em português, código e saída) e o texto que o narrador deve ler. O tempo de cada cena é uma estimativa a partir do tamanho da narração.

## Sumário

| #  | Início | Capítulo                       | Cena                           | Duração |
|----|--------|--------------------------------|--------------------------------|---------|
| 1  | 00:00  | —                              | Abertura                       | 18s     |
| 2  | 00:18  | Trabalhando com dados de texto | As etapas da construção        | 26s     |
| 3  | 00:44  | Trabalhando com dados de texto | Quebrando o texto em tokens    | 16s     |
| 4  | 01:00  | Trabalhando com dados de texto | O tokenizador do GPT-2         | 18s     |
| 5  | 01:18  | Trabalhando com dados de texto | Prevendo a próxima palavra     | 18s     |
| 6  | 01:36  | Trabalhando com dados de texto | De texto a vetores             | 23s     |
| 7  | 01:59  | Mecanismos de atenção          | Capítulo 3: atenção            | 15s     |
| 8  | 02:14  | Mecanismos de atenção          | Pontuações de atenção          | 20s     |
| 9  | 02:34  | Mecanismos de atenção          | Pesos e vetor de contexto      | 18s     |
| 10 | 02:52  | Mecanismos de atenção          | Query, key e value             | 20s     |
| 11 | 03:12  | Mecanismos de atenção          | Máscara causal                 | 19s     |
| 12 | 03:31  | Mecanismos de atenção          | Várias cabeças de atenção      | 19s     |
| 13 | 03:50  | Montando o modelo GPT          | Capítulo 4: a arquitetura      | 14s     |
| 14 | 04:04  | Montando o modelo GPT          | Normalização de camada         | 18s     |
| 15 | 04:22  | Montando o modelo GPT          | GELU e feed forward            | 20s     |
| 16 | 04:42  | Montando o modelo GPT          | Conexões de atalho             | 17s     |
| 17 | 04:59  | Montando o modelo GPT          | O bloco transformer            | 20s     |
| 18 | 05:19  | Montando o modelo GPT          | O modelo GPT completo          | 18s     |
| 19 | 05:37  | Montando o modelo GPT          | Gerando texto                  | 20s     |
| 20 | 05:57  | Pré-treinamento                | Capítulo 5: o ciclo de geração | 19s     |
| 21 | 06:16  | Pré-treinamento                | Medindo o erro                 | 17s     |
| 22 | 06:33  | Pré-treinamento                | Treino e overfitting           | 19s     |
| 23 | 06:52  | Pré-treinamento                | Pesos oficiais do GPT-2        | 18s     |
| 24 | 07:10  | Fine-tuning para classificação | Capítulo 6: classificação      | 13s     |
| 25 | 07:23  | Fine-tuning para classificação | Nova camada de saída           | 18s     |
| 26 | 07:41  | Fine-tuning para classificação | Treino e acurácia              | 19s     |

| #  | Início | Capítulo                       | Cena                     | Duração |
|----|--------|--------------------------------|--------------------------|---------|
| 27 | 08:00  | Fine-tuning para classificação | Classificando mensagens  | 14s     |
| 28 | 08:14  | Fine-tuning com instruções     | Capítulo 7: instruções   | 18s     |
| 29 | 08:32  | Fine-tuning com instruções     | Montando os lotes        | 19s     |
| 30 | 08:51  | Fine-tuning com instruções     | O modelo antes do ajuste | 16s     |
| 31 | 09:07  | Fine-tuning com instruções     | Treino e respostas       | 18s     |
| 32 | 09:25  | Fine-tuning com instruções     | O Llama 3 como avaliador | 19s     |
| 33 | 09:44  | —                              | Encerramento             | 16s     |

# Abertura

## Cena 1 — Abertura

00:00 → 00:18 · 18 segundos

### Transformer — construindo um LLM do zero

Este notebook reúne, em sequência, **todo o ciclo de construção de um modelo de linguagem do tipo GPT**, dos capítulos 2 a 7 do livro *Build a Large Language Model (From Scratch)*, de Sebastian Raschka ([código original](#), licença Apache 2.0).

| Capítulo | O que se constrói                                                                |
|----------|----------------------------------------------------------------------------------|
| 2        | Tokenização do texto, embeddings e o carregador de dados                         |
| 3        | O mecanismo de <b>atenção</b> : self-attention, máscara causal e multi-head      |
| 4        | A arquitetura <b>GPT</b> completa: LayerNorm, GELU, bloco transformer e o modelo |
| 5        | <b>Pré-treinamento</b> , avaliação e carregamento dos pesos oficiais do GPT-2    |
| 6        | <b>Fine-tuning</b> para classificar mensagens como spam                          |
| 7        | <b>Fine-tuning com instruções</b> , transformando o modelo em um assistente      |

**Como usar no Google Colab**

- Menu *Ambiente de execução* → *Alterar o tipo de ambiente de execução* → *GPU (T4)*.
- Execute primeiro a seção **Preparação do ambiente** logo abaixo.
- Depois execute os capítulos em ordem. Cada capítulo reaproveita o que foi construído nos anteriores.

**Sobre as saídas:** as saídas que já aparecem nas células vêm da execução original, feita numa GPU NVIDIA RTX 3060. Ao executar de novo, os números podem variar um pouco.

Antes de cada célula de código há uma nota em português (🇧🇷) dizendo, em poucas palavras, o que ela faz.

 **Na tela:** Título do notebook e a tabela com o que cada capítulo constrói.

 **Narração:** Nesta demonstração, vamos construir do zero um transformer, a arquitetura dos modelos de linguagem do tipo GPT, como o ChatGPT. A tabela mostra as

etapas da construção: preparar o texto, programar a atenção, montar a arquitetura, pré-treinar e fazer dois tipos de ajuste fino.

## Capítulo 2 — Trabalhando com dados de texto

### Cena 2 — As etapas da construção

00:18 → 00:44 · 26 segundos · 2 telas em sequência, dividindo o tempo entre elas

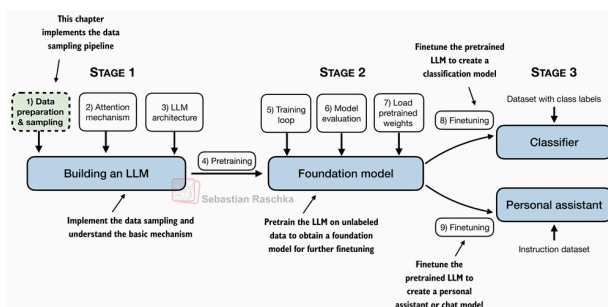
#### Tela A

### Capítulo 2 — Trabalhando com dados de texto

Neste capítulo damos o primeiro passo concreto na construção do nosso LLM: preparar os dados de texto. Vamos transformar texto bruto em **tokens**, converter esses tokens em **IDs** numéricos, usar o tokenizer **byte pair encoding** (BPE) do GPT-2 e montar um **dataloader** com janela deslizante que gera pares de entrada e alvo para a previsão da próxima palavra. Por fim, convertemos os IDs em vetores contínuos com **embeddings de tokens** e **embeddings de posição**. O resultado é exatamente a entrada que o mecanismo de atenção (capítulo 3) e o modelo GPT (capítulo 4) vão consumir.

#### Tela B

- Este capítulo trata da preparação e da amostragem dos dados, para deixar os dados de entrada "prontos" para o LLM



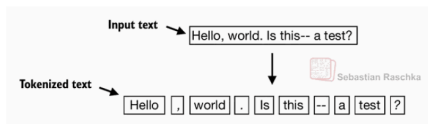
📺 **Na tela:** Tela A: título do capítulo 2. Tela B: o diagrama das três etapas (Stage 1, 2 e 3), com o passo 1 destacado em verde.

🗣️ **Narração:** Este diagrama mostra todas as etapas da construção de um LLM. Na etapa 1, construímos o modelo: preparamos os dados, programamos a atenção e montamos a arquitetura. Na etapa 2, pré-treinamos o modelo com texto comum e obtemos um modelo

base. Na etapa 3, fazemos o ajuste fino, criando um classificador e um assistente. O destaque verde mostra onde estamos: a preparação dos dados.

## Cena 3 — Quebrando o texto em tokens

00:44 → 01:00 · 16 segundos



**O que esta célula faz:** Tokeniza o conto inteiro e mostra os 30 primeiros tokens obtidos.

```
In [8]: preprocessed = re.split(r'([, . ? ! " ( ) \ ] | -- | \s)', raw_text)
preprocessed = [item.strip() for item in preprocessed if item.strip()]
print(preprocessed[:30])

['I', 'HAD', 'always', 'thought', 'Jack', 'Gisburn', 'rather', 'a', 'cheap', 'genius', '--', 'though', 'a', 'good', 'fell', 'ow', 'enough', '--', 'so', 'it', 'was', 'no', 'great', 'surprise', 'to', 'me', 'to', 'hear', 'that', ',', 'in']
```

**Na tela:** Diagrama: a frase "Hello, world. Is this-- a test?" dividida em pedaços. Abaixo, a célula que tokeniza o conto e mostra os 30 primeiros tokens.

**Narração:** Primeiro passo: a tokenização. O diagrama mostra a ideia: a frase Hello, world é quebrada em pedaços, palavras e pontuação, e cada pedaço é um token. Na célula, fazemos o mesmo com o conto inteiro e vemos os primeiros tokens.

## Cena 4 — O tokenizador do GPT-2

01:00 → 01:18 · 18 segundos

**O que esta célula faz:** Codifica um texto com o tokenizador BPE; até uma palavra inventada é convertida em IDs, sem erro.

```
In [28]: text = "Hello, do you like tea? <|endoftext|> In the sunlit terraces of someunknownPlace."
integers = tokenizer.encode(text, allowed_special={"<|endoftext|>"})
print(integers)

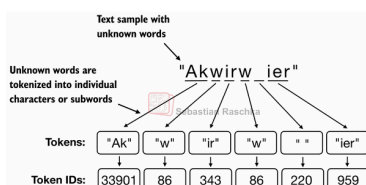
[15496, 11, 466, 345, 588, 8887, 30, 220, 50256, 554, 262, 4252, 18250, 8812, 2114, 286, 617, 34680, 27271, 13]
```

**O que esta célula faz:** Decodifica os IDs e recupera exatamente o texto original.

```
In [29]: strings = tokenizer.decode(integers)
print(strings)

Hello, do you like tea? <|endoftext|> In the sunlit terraces of someunknownPlace.
```

- Tokenizers BPE quebram palavras desconhecidas em subpalavras e caracteres individuais:



 **Na tela:** Célula que converte uma frase em números (IDs); célula que converte os números de volta no mesmo texto; diagrama da palavra inventada "Akwirw ier" quebrada em pedaços.

 **Narração:** O GPT-2 usa um tokenizador chamado byte pair encoding. A primeira célula transforma uma frase em números, os IDs; a segunda reconstrói o texto original. O diagrama mostra o truque: uma palavra desconhecida, como Akwirw ier, é quebrada em pedaços que existem no vocabulário.

## Cena 5 — Prevendo a próxima palavra

01:18 → 01:36 · 18 segundos

- Passo a passo, a previsão ficaria assim:

 **O que esta célula faz:** Mostra, passo a passo, qual token o modelo deve prever a partir de cada contexto.

```
In [33]: for i in range(1, context_size+1):
          context = enc_sample[:i]
          desired = enc_sample[i]

          print(context, "---->", desired)
```

```
[290] ----> 4920
[290, 4920] ----> 2241
[290, 4920, 2241] ----> 287
[290, 4920, 2241, 287] ----> 257
```

 **O que esta célula faz:** Repete a demonstração anterior, agora mostrando os tokens como texto legível.

```
In [34]: for i in range(1, context_size+1):
          context = enc_sample[:i]
          desired = enc_sample[i]

          print(tokenizer.decode(context), "---->", tokenizer.decode([desired]))

and ----> established
and established ----> himself
and established himself ----> in
and established himself in ----> a
```

 **Na tela:** Duas células. A primeira mostra, com números, qual token vem depois de cada trecho. A segunda mostra o mesmo em texto: "and ----> established" e assim por diante.

 **Narração:** É assim que um LLM aprende: prevendo a próxima palavra. A primeira saída mostra isso com números: depois do token 290 vem o 4920, e assim por diante. A

segunda mostra o mesmo em texto: depois de and vem established; depois de and established, himself.

## Cena 6 — De texto a vetores

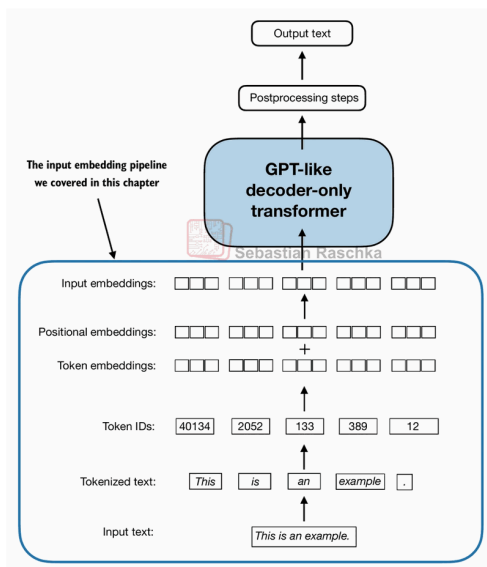
01:36 → 01:59 · 23 segundos

 **O que esta célula faz:** Soma os embeddings de token e de posição, obtendo os embeddings de entrada do LLM.

```
In [53]: input_embeddings = token_embeddings + pos_embeddings
print(input_embeddings.shape)

torch.Size([8, 4, 256])
```

- Na fase inicial do fluxo de processamento da entrada, o texto de entrada é segmentado em tokens separados
- Após essa segmentação, esses tokens são transformados em token IDs com base em um vocabulário predefinido:



 **Na tela:** Célula que soma os dois embeddings, com formato [8, 4, 256]. Abaixo, o diagrama do fluxo de entrada, de baixo (Input text) para cima (GPT).

 **Narração:** Leia o diagrama de baixo para cima. O texto vira tokens, os tokens viram IDs, e cada ID vira um vetor, o embedding do token. Somamos a ele um embedding de

posição, que indica onde o token está. Essa soma é a entrada do modelo: na célula, oito textos, de quatro tokens, com vetores de 256 números.

## Capítulo 3 — Mecanismos de atenção

### Cena 7 — Capítulo 3: atenção

01:59 → 02:14 · 15 segundos

#### Capítulo 3: Programando mecanismos de atenção

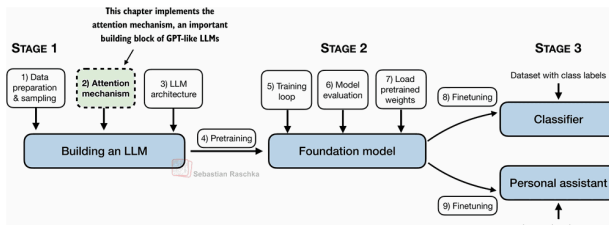
Pacotes usados neste notebook:

🇧🇷 O que esta célula faz: Mostra a versão do PyTorch instalada no ambiente.

```
In [54]: from importlib.metadata import version
print("torch version:", version("torch"))
```

torch version: 2.3.1

- Este capítulo trata dos mecanismos de atenção, o motor dos LLMs:

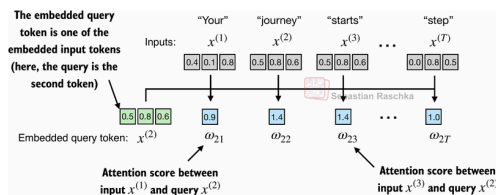


**Na tela:** Título do capítulo 3 e o mapa das etapas, agora com o passo 2 (Attention mechanism) destacado.

**Narração:** Capítulo 3. No mapa, o destaque passa para o passo 2: a atenção, a peça central de um LLM. É ela que permite a cada palavra olhar para as outras e decidir quais importam para o seu significado.

## Cena 8 — Pontuações de atenção

02:14 → 02:34 · 20 segundos



- Usamos o elemento 2 da sequência de entrada,  $x^{(2)}$ , como exemplo para calcular o context vector  $z^{(2)}$ ; mais adiante nesta seção, vamos generalizar isso para calcular todos os context vectors.
- O primeiro passo é calcular as pontuações de atenção não normalizadas, fazendo o produto escalar entre a query  $x^{(2)}$  e todos os outros tokens de entrada:

**O que esta célula faz:** Usa o segundo token como query e calcula sua pontuação de atenção em relação a cada um dos seis tokens por meio de produtos escalares.

```
In [56]: query = inputs[1] # 2nd input token is the query

attn_scores_2 = torch.empty(inputs.shape[0])
for i, x_i in enumerate(inputs):
    attn_scores_2[i] = torch.dot(x_i, query) # dot product (transpose not necessary here since they are 1-dim vectors)

print(attn_scores_2)

tensor([0.9544, 1.4950, 1.4754, 0.8434, 0.7070, 1.0865])
```

**Na tela:** Diagrama: a frase "Your journey starts with one step", cada palavra como um vetor; "journey" é a consulta (query) e gera uma pontuação para cada palavra. Abaixo, a célula com as seis pontuações.

**Narração:** Começamos com uma versão simples. No diagrama, cada palavra de Your journey starts with one step é um vetor de três números. Journey é a consulta, e a

comparamos com cada palavra por um produto escalar. A saída mostra as seis pontuações; as maiores são as de journey e starts.

## Cena 9 — Pesos e vetor de contexto

02:34 → 02:52 · 18 segundos

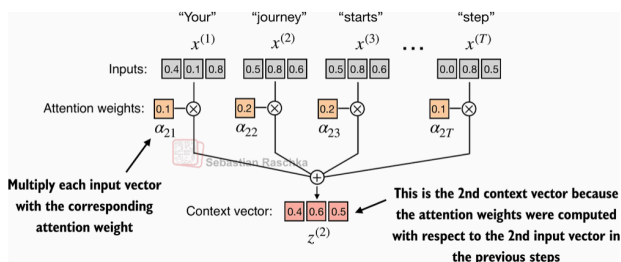
 **O que esta célula faz:** Aplica a softmax do PyTorch às pontuações e obtém os mesmos pesos de atenção da versão manual, somando 1.

```
In [60]: attn_weights_2 = torch.softmax(attn_scores_2, dim=0)
```

```
print("Attention weights:", attn_weights_2)
print("Sum:", attn_weights_2.sum())
```

Attention weights: tensor([0.1385, 0.2379, 0.2333, 0.1240, 0.1082, 0.1581])  
Sum: tensor(1.)

- **Passo 3:** calcular o context vector  $z^{(2)}$  multiplicando os tokens de entrada (já em forma de embedding),  $x^{(i)}$ , pelos pesos de atenção e somando os vetores resultantes:



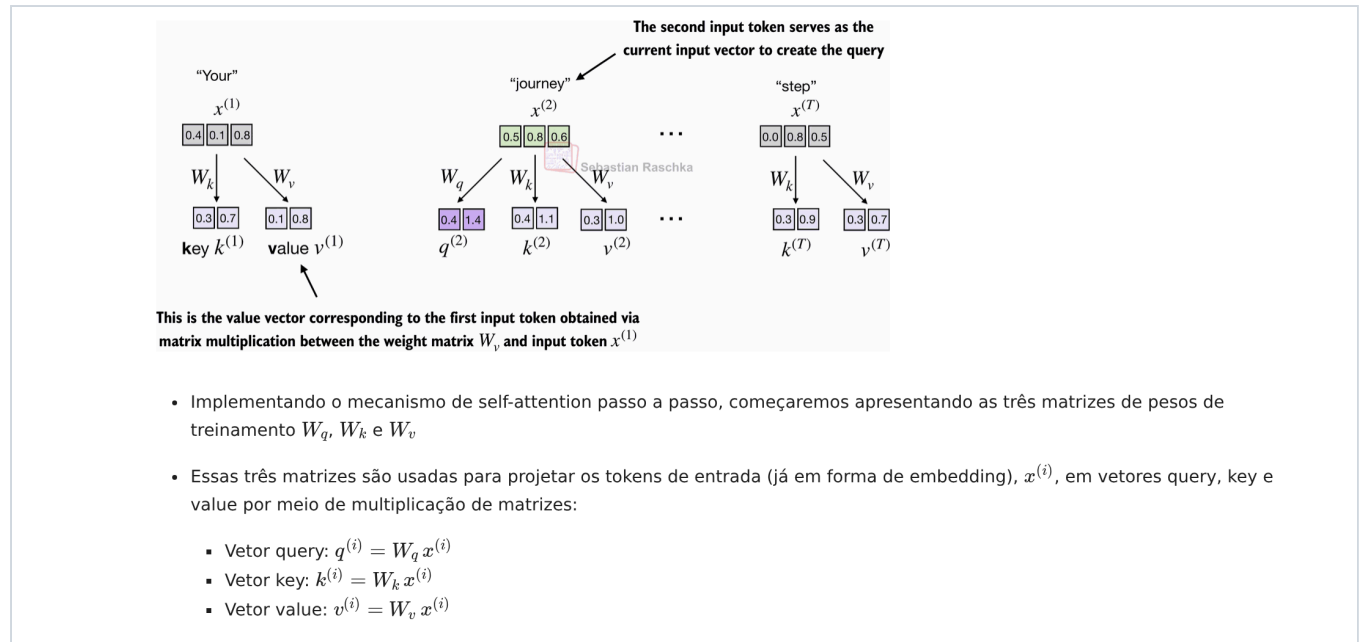
 **Na tela:** Célula da softmax: os pesos de atenção e a soma igual a 1. Abaixo, o diagrama que multiplica cada palavra pelo seu peso e soma tudo, gerando o vetor de contexto.

 **Narração:** A softmax transforma as pontuações em pesos positivos que somam 1, como mostra a saída. No diagrama, multiplicamos o vetor de cada palavra pelo seu peso e

somamos tudo. O resultado é o vetor de contexto de journey, que mistura informação da frase inteira.

## Cena 10 — Query, key e value

02:52 → 03:12 · 20 segundos



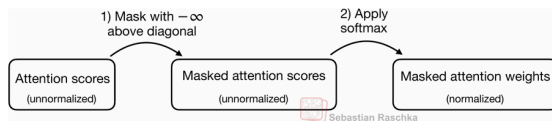
**Na tela:** Diagrama: cada palavra passa por três matrizes de pesos ( $W_q$ ,  $W_k$ ,  $W_v$ ) e gera três vetores: query, key e value. Abaixo, as fórmulas.

**Narração:** Agora a versão real, com pesos que o modelo aprende. No diagrama, cada palavra passa por três matrizes e gera três vetores: a query, que é a pergunta; a key, uma

etiqueta; e o value, o conteúdo. A query de journey é comparada com as keys de todas as palavras.

## Cena 11 — Máscara causal

03:12 → 03:31 · 19 segundos



**O que esta célula faz:** Preenche as pontuações acima da diagonal com menos infinito, uma forma mais eficiente de aplicar a máscara causal.

```
In [82]: mask = torch.triu(torch.ones(context_length, context_length), diagonal=1)
masked = attn_scores.masked_fill(mask.bool(), -torch.inf)
print(masked)

tensor([[0.2899,  -inf,  -inf,  -inf,  -inf,  -inf],
        [0.4656, 0.1723, -inf,  -inf,  -inf,  -inf],
        [0.4594, 0.1703, 0.1731, -inf,  -inf,  -inf],
        [0.2642, 0.1024, 0.1036, 0.0186, -inf,  -inf],
        [0.2183, 0.0874, 0.0882, 0.0177, 0.0786, -inf],
        [0.3408, 0.1270, 0.1290, 0.0198, 0.1290, 0.0078]],
      grad_fn=<MaskedFillBackward0>)
```

- Como vemos abaixo, agora os pesos de atenção de cada linha voltam a somar 1 corretamente:

**O que esta célula faz:** Aplica a softmax às pontuações mascaradas; os valores futuros viram zero e cada linha já soma 1, sem renormalização.

```
In [83]: attn_weights = torch.softmax(masked / keys.shape[-1]**0.5, dim=-1)
print(attn_weights)

tensor([[1.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.5517, 0.4483, 0.0000, 0.0000, 0.0000, 0.0000],
        [0.3800, 0.3097, 0.3103, 0.0000, 0.0000, 0.0000],
        [0.2758, 0.2460, 0.2462, 0.2319, 0.0000, 0.0000],
        [0.2175, 0.1983, 0.1984, 0.1888, 0.1971, 0.0000],
        [0.1935, 0.1663, 0.1666, 0.1542, 0.1666, 0.1529]],
      grad_fn=<SoftmaxBackward0>)
```

**Na tela:** Diagrama em dois passos: 1) máscara com menos infinito acima da diagonal; 2) softmax. Abaixo, a matriz com -inf e a matriz final de pesos, com zeros acima da diagonal.

**Narração:** Ao gerar texto, o modelo não pode olhar para o futuro. O diagrama mostra dois passos: primeiro, menos infinito acima da diagonal, como na primeira saída; depois, a

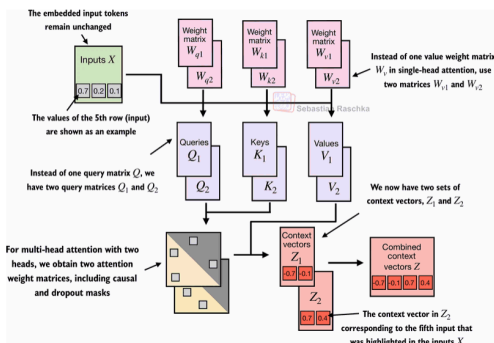
softmax, que transforma esses valores em zero. Na segunda saída, cada palavra só olha para si e para as anteriores.

## Cena 12 — Várias cabeças de atenção

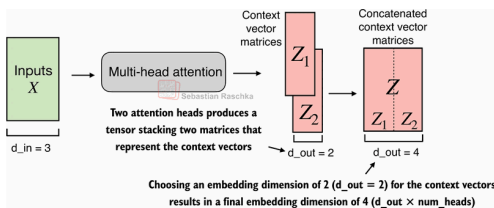
03:31 → 03:50 · 19 segundos · 2 telas em sequência, dividindo o tempo entre elas

### Tela A

- Abaixo está um resumo da self-attention implementada anteriormente (as máscaras causal e de dropout não aparecem, para simplificar)
- Isso também é chamado de single-head attention (atenção de uma única cabeça):



- Basta empilhar vários módulos de atenção de uma cabeça para obter um módulo de multi-head attention (atenção com múltiplas cabeças):



- A ideia principal da multi-head attention é executar o mecanismo de atenção várias vezes (em paralelo), com projeções lineares diferentes e aprendidas. Isso permite que o modelo preste atenção, ao mesmo tempo, a informações de diferentes subespaços de representação em diferentes posições.

**🇧🇷 O que esta célula faz:** Cria uma multi-head attention empilhando duas cabeças de atenção causal e concatenando suas saídas, que passam a ter 4 dimensões por token.

## Tela B

```
In [88]: class MultiHeadAttentionWrapper(nn.Module):

    def __init__(self, d_in, d_out, context_length, dropout, num_heads, qkv_bias=False):
        super().__init__()
        self.heads = nn.ModuleList(
            [CausalAttention(d_in, d_out, context_length, dropout, qkv_bias)
             for _ in range(num_heads)]
        )

    def forward(self, x):
        return torch.cat([head(x) for head in self.heads], dim=-1)

torch.manual_seed(123)

context_length = batch.shape[1] # This is the number of tokens
d_in, d_out = 3, 2
mha = MultiHeadAttentionWrapper(d_in, d_out, context_length, 0.0, num_heads=2)

context_vecs = mha(batch)

print(context_vecs)
print("context_vecs.shape:", context_vecs.shape)

tensor([[[[-0.4519,  0.2216,  0.4772,  0.1063],
          [-0.5874,  0.0058,  0.5891,  0.3257],
          [-0.6300, -0.0632,  0.6202,  0.3860],
          [-0.5675, -0.0843,  0.5478,  0.3589],
          [-0.5526, -0.0981,  0.5321,  0.3428],
          [-0.5299, -0.1081,  0.5077,  0.3493]],

         [[[-0.4519,  0.2216,  0.4772,  0.1063],
          [-0.5874,  0.0058,  0.5891,  0.3257],
          [-0.6300, -0.0632,  0.6202,  0.3860],
          [-0.5675, -0.0843,  0.5478,  0.3589],
          [-0.5526, -0.0981,  0.5321,  0.3428],
          [-0.5299, -0.1081,  0.5077,  0.3493]]], grad_fn=<CatBackward0>)]
context_vecs.shape: torch.Size([2, 6, 4])
```

 **Na tela:** Tela A: dois diagramas: duas cabeças de atenção em paralelo gerando Z1 e Z2, e a concatenação em Z. Tela B: a célula da multi-head attention, com formato de saída [2, 6, 4].

 **Narração:** Por fim, a multi-head attention. O primeiro diagrama mostra duas cabeças de atenção em paralelo, cada uma com as suas matrizes, gerando os vetores Z1 e Z2. O

segundo mostra que eles são concatenados: duas cabeças de tamanho 2 geram vetores de tamanho 4, como confirma a saída.

## Capítulo 4 — Montando o modelo GPT

### Cena 13 — Capítulo 4: a arquitetura

03:50 → 04:04 · 14 segundos

#### Capítulo 4: Implementando um modelo GPT do zero para gerar texto

 **O que esta célula faz:** Importa as bibliotecas usadas no capítulo e mostra as versões instaladas de matplotlib, PyTorch e tiktoken.

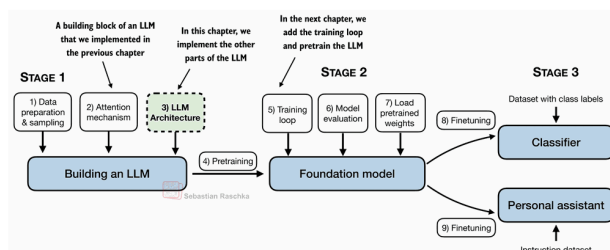
```
In [92]: from importlib.metadata import version

import matplotlib
import tiktoken
import torch

print("matplotlib version:", version("matplotlib"))
print("torch version:", version("torch"))
print("tiktoken version:", version("tiktoken"))
```

matplotlib version: 3.8.4  
torch version: 2.3.1  
tiktoken version: 0.7.0

- Neste capítulo, implementamos uma arquitetura de LLM no estilo GPT; o próximo capítulo vai se concentrar no treinamento desse LLM

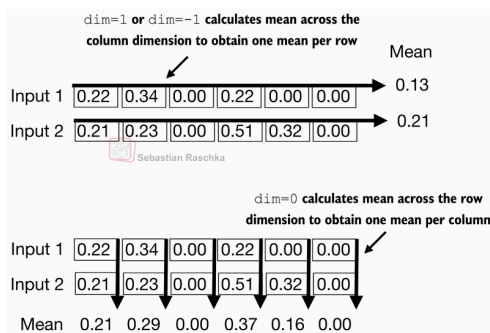


**Na tela:** Título do capítulo 4 e o mapa das etapas com o passo 3 (LLM Architecture) destacado.

**Narração:** Capítulo 4. No mapa, estamos no passo 3: a arquitetura. A atenção já está pronta; agora construímos as outras peças e juntamos tudo num modelo GPT completo. O treino fica para o próximo capítulo.

## Cena 14 — Normalização de camada

04:04 → 04:22 · 18 segundos



- Subtrair a média e dividir pela raiz quadrada da variância (o desvio padrão) centraliza as entradas para que tenham média 0 e variância 1 ao longo da dimensão das colunas (features):

**O que esta célula faz:** Normaliza as saídas subtraindo a média e dividindo pelo desvio padrão; agora cada linha tem média 0 e variância 1.

```
In [99]: out_norm = (out - mean) / torch.sqrt(var)
print("Normalized layer outputs:\n", out_norm)

mean = out_norm.mean(dim=-1, keepdim=True)
var = out_norm.var(dim=-1, keepdim=True)
print("Mean:\n", mean)
print("Variance:\n", var)

Normalized layer outputs:
tensor([[ 0.6159,  1.4126, -0.8719,  0.5872, -0.8719, -0.8719],
        [-0.0189,  0.1121, -1.0876,  1.5173,  0.5647, -1.0876]],
        grad_fn=<DivBackward0>)

Mean:
tensor([[ -0.0000],
        [ -0.0000]], grad_fn=<MeanBackward1>)

Variance:
tensor([[1.0000],
        [1.0000]], grad_fn=<VarBackward0>)
```

**Na tela:** Diagrama: média calculada por linha (um valor por token) e por coluna. Abaixo, a célula que normaliza as saídas, com média 0 e variância 1.

**Narração:** Primeira peça: a normalização de camada. O diagrama mostra a diferença entre tirar a média por linha, isto é, por token, ou por coluna; usamos por linha. Na célula,

subtraímos a média e dividimos pelo desvio padrão, e a saída confirma média zero e variância um.

## Cena 15 — GELU e feed forward

04:22 → 04:42 · 20 segundos · 2 telas em sequência, dividindo o tempo entre elas

### Tela A

 **O que esta célula faz:** Desenha lado a lado os gráficos da GELU e da ReLU para comparar as duas funções de ativação.

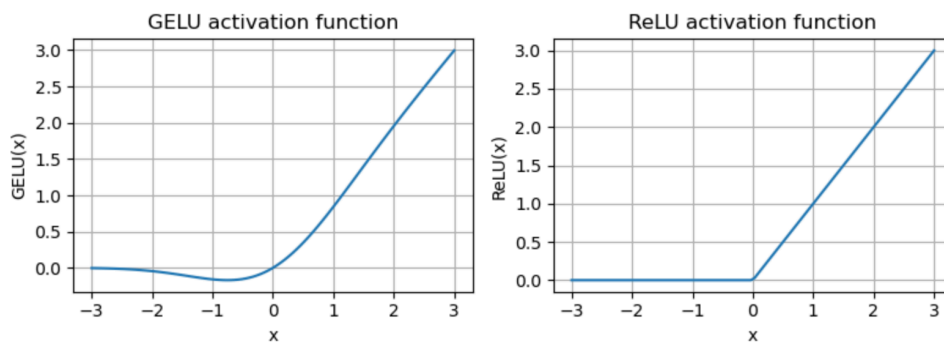
```
In [105... import matplotlib.pyplot as plt

gelu, relu = GELU(), nn.ReLU()

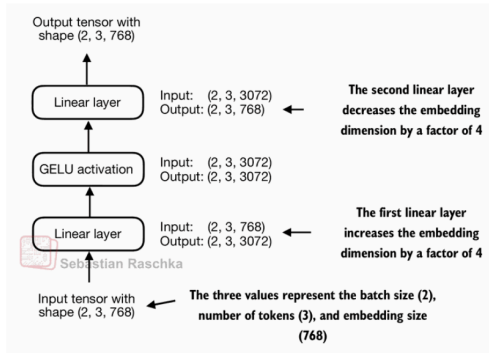
# Some sample data
x = torch.linspace(-3, 3, 100)
y_gelu, y_relu = gelu(x), relu(x)

plt.figure(figsize=(8, 3))
for i, (y, label) in enumerate(zip([y_gelu, y_relu], ["GELU", "ReLU"]), 1):
    plt.subplot(1, 2, i)
    plt.plot(x, y)
    plt.title(f"{label} activation function")
    plt.xlabel("x")
    plt.ylabel(f"{label}(x)")
    plt.grid(True)

plt.tight_layout()
plt.show()
```



## Tela B



**O que esta célula faz:** Passa um tensor aleatório pelo FeedForward; a saída mantém exatamente o formato da entrada, [2, 3, 768].

```
In [108... ffn = FeedForward(GPT_CONFIG_124M)

# input shape: [batch_size, num_token, emb_size]
x = torch.rand(2, 3, 768)
out = ffn(x)
print(out.shape)

torch.Size([2, 3, 768])
```

**Na tela:** Tela A: gráfico comparando GELU e ReLU e a célula da classe FeedForward. Tela B: diagrama da rede feed forward (768 → 3072 → 768) e a saída [2, 3, 768].

**Narração:** Segunda peça: a rede feed forward. O gráfico compara sua ativação, a GELU, com a ReLU: a GELU é uma curva suave. O diagrama mostra a rede: uma camada

expande cada vetor de 768 para 3072 números e outra volta para 768, e a saída mantém o formato da entrada.

## Cena 16 — Conexões de atalho

04:42 → 04:59 · 17 segundos

 **O que esta célula faz:** Calcula os gradientes da rede sem shortcut connections: eles ficam minúsculos nas primeiras camadas, sinal de desaparecimento do gradiente.

```
In [110... layer_sizes = [3, 3, 3, 3, 3, 1]

sample_input = torch.tensor([[1., 0., -1.]])

torch.manual_seed(123)
model_without_shortcut = ExampleDeepNeuralNetwork(
    layer_sizes, use_shortcut=False
)
print_gradients(model_without_shortcut, sample_input)

layers.0.0.weight has gradient mean of 0.00020173584925942123
layers.1.0.weight has gradient mean of 0.00012011159560643137
layers.2.0.weight has gradient mean of 0.0007152040489017963
layers.3.0.weight has gradient mean of 0.0013988736318424344
layers.4.0.weight has gradient mean of 0.005049645435065031
```

- Em seguida, vamos imprimir os valores dos gradientes **com** shortcut connections:

 **O que esta célula faz:** Calcula os gradientes da mesma rede com shortcut connections: agora eles permanecem em um tamanho saudável em todas as camadas.

```
In [111... torch.manual_seed(123)
model_with_shortcut = ExampleDeepNeuralNetwork(
    layer_sizes, use_shortcut=True
)
print_gradients(model_with_shortcut, sample_input)

layers.0.0.weight has gradient mean of 0.22169791162014008
layers.1.0.weight has gradient mean of 0.20694105327129364
layers.2.0.weight has gradient mean of 0.32896995544433594
layers.3.0.weight has gradient mean of 0.2665732204914093
layers.4.0.weight has gradient mean of 1.3258540630340576
```

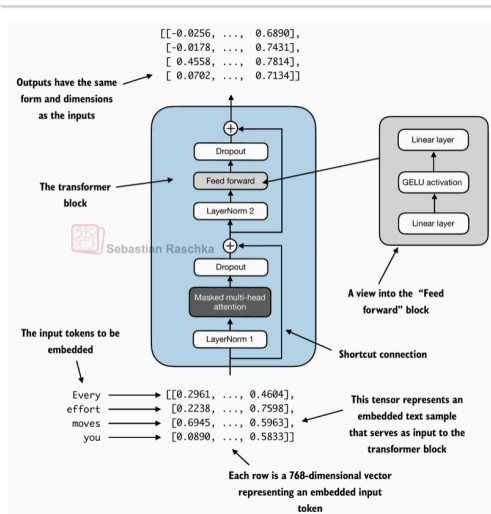
 **Na tela:** Duas células com os gradientes de uma rede de cinco camadas: sem atalhos (valores minúsculos) e com atalhos (valores saudáveis).

 **Narração:** Terceira peça: os atalhos, ou shortcut connections. Sem atalhos, os gradientes das primeiras camadas são minúsculos, cerca de 0,0002, e elas quase não

aprendem. Com atalhos, que somam a entrada de cada camada à sua saída, os gradientes ficam perto de 0,2.

## Cena 17 — O bloco transformer

04:59 → 05:19 · 20 segundos



- Suponha que temos 2 amostras de entrada com 6 tokens cada, em que cada token é um vetor de embedding de 768 dimensões; então este transformer block aplica self-attention (autoatenção), seguida de camadas lineares, para produzir uma saída de tamanho semelhante
- Você pode pensar na saída como uma versão aprimorada dos vetores de contexto que discutimos no capítulo anterior

**O que esta célula faz:** Passa um tensor aleatório por um transformer block; a saída tem o mesmo formato da entrada, [2, 4, 768].

```
In [113]: torch.manual_seed(123)

x = torch.rand(2, 4, 768) # Shape: [batch_size, num_tokens, emb_dim]
block = TransformerBlock(GPT_CONFIG_124M)
output = block(x)

print("Input shape:", x.shape)
print("Output shape:", output.shape)

Input shape: torch.Size([2, 4, 768])
Output shape: torch.Size([2, 4, 768])
```

**Na tela:** Diagrama do bloco transformer (lido de baixo para cima), com o detalhe da rede feed forward à direita. Abaixo, a célula com entrada e saída de formato [2, 4, 768].

**Narração:** Juntamos as peças no bloco transformer. Leia o diagrama de baixo para cima: normalização, atenção com máscara, dropout e um atalho que soma a entrada de

volta; depois, nova normalização, feed forward, dropout e outro atalho. A saída tem o mesmo formato da entrada, e por isso podemos empilhar blocos.

## Cena 18 — O modelo GPT completo

05:19 → 05:37 · 18 segundos

 **O que esta célula faz:** Conta o total de parâmetros do modelo: cerca de 163 milhões.

```
In [116... total_params = sum(p.numel() for p in model.parameters())
print(f"Total number of parameters: {total_params:,}")
```

Total number of parameters: 163,009,536

- Como vemos acima, este modelo tem 163M de parâmetros, e não 124M; por quê?
- No artigo original do GPT-2, os pesquisadores aplicaram weight tying (compartilhamento de pesos), o que significa que reutilizaram a camada de embedding de tokens ( `tok_emb` ) como camada de saída, ou seja, definiram `self.out_head.weight = self.tok_emb.weight`
- A camada de embedding de tokens projeta os tokens de entrada, codificados em one-hot com 50.257 dimensões, em uma representação de embedding de 768 dimensões
- A camada de saída projeta os embeddings de 768 dimensões de volta para uma representação de 50.257 dimensões, para que possamos convertê-los de volta em palavras (mais sobre isso na próxima seção)
- Assim, a camada de embedding e a camada de saída têm o mesmo número de parâmetros de peso, como podemos ver pelo formato de suas matrizes de pesos: o próximo capítulo
- Mas, antes, uma observação rápida sobre o seu tamanho: nós nos referimos a ele como um modelo de 124M de parâmetros; podemos conferir esse número da seguinte forma:

 **O que esta célula faz:** Mostra que a camada de embedding de tokens e a camada de saída têm matrizes de pesos do mesmo formato, [50257, 768].

```
In [117... print("Token embedding layer shape:", model.tok_emb.weight.shape)
print("Output layer shape:", model.out_head.weight.shape)
```

Token embedding layer shape: torch.Size([50257, 768])  
Output layer shape: torch.Size([50257, 768])

- No artigo original do GPT-2, os pesquisadores reutilizaram a matriz de embedding de tokens como matriz de saída
- Da mesma forma, se subtrairmos o número de parâmetros da camada de saída, obteremos um modelo de 124M de parâmetros:

 **O que esta célula faz:** Desconta os parâmetros da camada de saída (como no weight tying do GPT-2) e chega a cerca de 124 milhões de parâmetros.

```
In [118... total_params_gpt2 = total_params - sum(p.numel() for p in model.out_head.parameters())
print(f"Number of trainable parameters considering weight tying: {total_params_gpt2:,}")
```

Number of trainable parameters considering weight tying: 124,412,160

 **Na tela:** Células que contam os parâmetros: 163.009.536 no total; as matrizes de embedding e de saída com o mesmo formato; 124.412.160 descontando a camada de saída.

 **Narração:** Empilhando doze blocos, com embeddings na entrada e uma camada de saída no final, temos o GPT completo: 163 milhões de parâmetros. O GPT-2 original

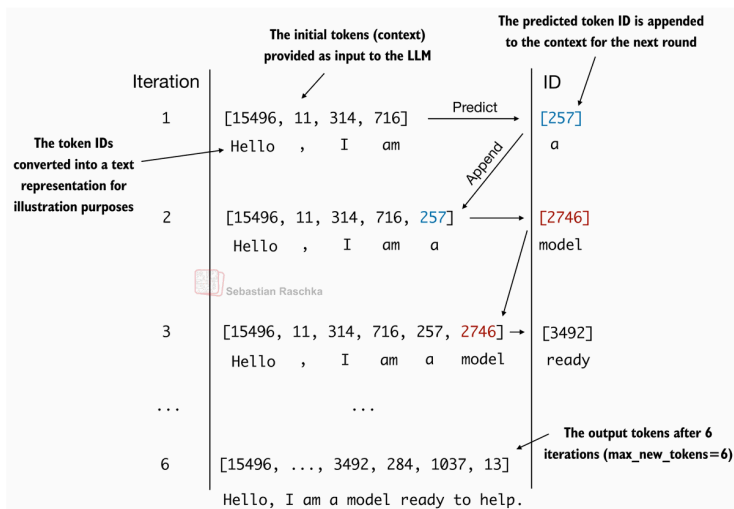
reaproveita a matriz de embedding na saída, pois as duas têm o mesmo formato. Descontando uma delas, chegamos aos 124 milhões.

## Cena 19 — Gerando texto

05:37 → 05:57 · 20 segundos · 2 telas em sequência, dividindo o tempo entre elas

### Tela A

- A função `generate_text_simple` acima implementa um processo iterativo, no qual cria um token de cada vez



### Tela B

**O que esta célula faz:** Converte os IDs gerados de volta em texto; como o modelo ainda não foi treinado, as palavras adicionadas não fazem sentido.

```
In [123... decoded_text = tokenizer.decode(out.squeeze(0).tolist())  
print(decoded_text)
```

Hello, I am Featureiman Byeswickattribution argue

- Observe que o modelo não foi treinado; por isso os textos de saída aleatórios acima
- Vamos treinar o modelo no próximo capítulo

**Na tela:** Tela A: diagrama da geração em rodadas: "Hello, I am" → a → model → ready. Tela B: a saída do nosso modelo sem treino, sem sentido.

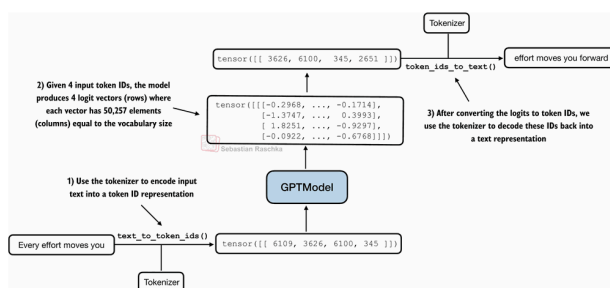
**Narração:** Como o modelo gera texto? No diagrama, partimos de Hello, I am; o modelo prevê o próximo token, a, que é acrescentado ao final, e o processo se repete, um token

por vez. Mas o nosso modelo ainda tem pesos aleatórios, e a saída não faz sentido. Falta treinar.

## Capítulo 5 — Pré-treinamento

### Cena 20 — Capítulo 5: o ciclo de geração

05:57 → 06:16 · 19 segundos



**O que esta célula faz:** Define funções para converter texto em IDs de tokens e vice-versa e gera 10 tokens a partir de "Every effort moves you"; como o modelo ainda não foi treinado, a saída é um amontoado sem sentido.

```
In [126... import tiktoken
from suporte.previous_chapters_cap05 import generate_text_simple

def text_to_token_ids(text, tokenizer):
    encoded = tokenizer.encode(text, allowed_special={'<|endoftext|>'})
    encoded_tensor = torch.tensor(encoded).unsqueeze(0) # add batch dimension
    return encoded_tensor

def token_ids_to_text(token_ids, tokenizer):
    flat = token_ids.squeeze(0) # remove batch dimension
    return tokenizer.decode(flat.tolist())

start_context = "Every effort moves you"
tokenizer = tiktoken.get_encoding("gpt2")

token_ids = generate_text_simple(
    model=model,
    idx=text_to_token_ids(start_context, tokenizer),
    max_new_tokens=10,
    context_size=GPT_CONFIG_124M["context_length"]
)

print("Output text:\n", token_ids_to_text(token_ids, tokenizer))
```

Output text:  
Every effort moves you rentingetic wasn, refres RexMeChicular stren

**Na tela:** Diagrama em três passos: texto → IDs, modelo → pontuações para cada palavra do vocabulário, IDs → texto. Abaixo, a célula que gera texto com o modelo ainda sem treino.

**Narração:** Capítulo 5: o treino. O diagrama resume a geração em três passos: o tokenizador transforma o texto em IDs; o modelo dá uma pontuação para cada uma das

50.257 palavras do vocabulário; e os IDs escolhidos voltam a ser texto. Sem treino, o resultado não tem sentido.

## Cena 21 — Medindo o erro

06:16 → 06:33 · 17 segundos

 **O que esta célula faz:** Calcula a cross entropy com a função pronta do PyTorch e obtém o mesmo valor de 10,77 calculado passo a passo.

```
In [137... loss = torch.nn.functional.cross_entropy(logits_flat, targets_flat)
print(loss)
tensor(10.7722)
```

- Um conceito relacionado à cross entropy loss é a perplexity (perplexidade) de um LLM
- A perplexity é simplesmente a exponencial da cross entropy loss

 **O que esta célula faz:** Calcula a perplexity como a exponencial da loss: cerca de 47.679, ou seja, o modelo está tão incerto quanto se escolhesse entre dezenas de milhares de tokens.

```
In [138... perplexity = torch.exp(loss)
print(perplexity)
tensor(47678.8633)
```

📺 **Na tela:** Célula da cross entropy (10,7722) e célula da perplexidade (47.678).

🗣️ **Narração:** Para treinar, medimos o quanto o modelo erra: essa é a loss, calculada com a cross entropy, que aqui vale 10,77. A perplexidade dá uma leitura intuitiva: 47 mil significa que o modelo está tão indeciso quanto se sorteasse entre 47 mil palavras.

## Cena 22 — Treino e overfitting

06:33 → 06:52 · 19 segundos

🇧🇷 **O que esta célula faz:** Plota as curvas de loss de treino e de validação ao longo das épocas e dos tokens vistos; a validação estaciona bem acima do treino, sinal de overfitting.

```
In [151... import matplotlib.pyplot as plt

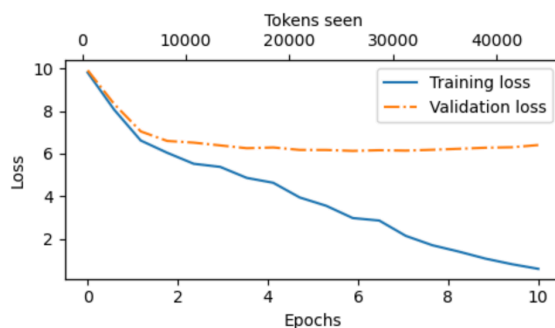
def plot_losses(epochs_seen, tokens_seen, train_losses, val_losses):
    fig, ax1 = plt.subplots(figsize=(5, 3))

    # Plot training and validation loss against epochs
    ax1.plot(epochs_seen, train_losses, label="Training loss")
    ax1.plot(epochs_seen, val_losses, linestyle="-. ", label="Validation loss")
    ax1.set_xlabel("Epochs")
    ax1.set_ylabel("Loss")
    ax1.legend(loc="upper right")

    # Create a second x-axis for tokens seen
    ax2 = ax1.twinx() # Create a second x-axis that shares the same y-axis
    ax2.plot(tokens_seen, train_losses, alpha=0) # Invisible plot for aligning ticks
    ax2.set_xlabel("Tokens seen")

    fig.tight_layout() # Adjust layout to make room
    plt.savefig("loss-plot.pdf")
    plt.show()

epochs_tensor = torch.linspace(0, num_epochs, len(train_losses))
plot_losses(epochs_tensor, tokens_seen, train_losses, val_losses)
```



- Olhando os resultados acima, vemos que o modelo começa gerando sequências de palavras incompreensíveis, enquanto, perto do final, consegue produzir frases gramaticalmente mais ou menos corretas
- No entanto, com base nas losses dos conjuntos de treino e de validação, percebemos que o modelo começa a sofrer overfitting (sobreajuste)
- Se examinássemos alguns trechos que ele escreve perto do final, descobriríamos que eles aparecem literalmente no conjunto de treino: o modelo simplesmente memoriza os dados de treino
- Mais adiante, veremos estratégias de decodificação que podem atenuar essa memorização até certo ponto
- Observe que o overfitting aqui ocorre porque temos um conjunto de treino muito, muito pequeno e iteramos sobre ele muitas vezes
  - O treinamento do LLM aqui tem finalidade principalmente didática; queremos, sobretudo, ver que o modelo consegue aprender a produzir texto coerente
  - Em vez de passar semanas ou meses treinando este modelo em grandes quantidades de hardware caro, vamos carregar pesos pré-treinados mais adiante

 **Na tela:** Célula que desenha o gráfico e o gráfico das losses: treino (azul) caindo até perto de zero, validação (laranja) parando perto de 6.

 **Narração:** O gráfico mostra dez épocas de treino. A loss de treino, em azul, cai de quase 10 para menos de 1; a de validação, em laranja, para perto de 6. Isso é overfitting: treinando com um único conto, o modelo decora o texto em vez de generalizar.

## Cena 23 — Pesos oficiais do GPT-2

06:52 → 07:10 · 18 segundos

- Se o modelo foi carregado corretamente, podemos usá-lo para gerar texto novo com a nossa função `generate` anterior:

 **O que esta célula faz:** Gera texto com o GPT-2 carregado a partir de "Every effort moves you"; o resultado é inglês fluente e coerente, sinal de que os pesos foram carregados corretamente.

```
In [179... torch.manual_seed(123)

token_ids = generate(
    model=gpt,
    idx=text_to_token_ids("Every effort moves you", tokenizer),
    max_new_tokens=25,
    context_size=NEW_CONFIG["context_length"],
    top_k=50,
    temperature=1.5
)

print("Output text:\n", token_ids_to_text(token_ids, tokenizer))
```

Output text:

Every effort moves you toward an equal share for each vote plus half. Inequality is often not an accurate representation of human worth; to know the

 **Na tela:** Célula que gera texto com os pesos do GPT-2 carregados; a saída é um texto fluente em inglês.

 **Narração:** Treinar um LLM de verdade exige bilhões de palavras. Por isso, carregamos no nosso modelo os pesos oficiais do GPT-2, da OpenAI. Como a arquitetura é a mesma,

eles se encaixam, e o modelo agora continua Every effort moves you com um texto fluente.

## Capítulo 6 — Fine-tuning para classificação

### Cena 24 — Capítulo 6: classificação

07:10 → 07:23 · 13 segundos

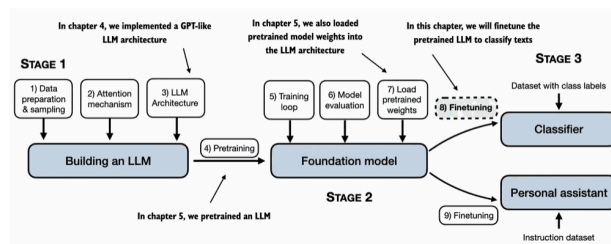
#### Capítulo 6: Finetuning para classificação de textos

**O que esta célula faz:** Confere as versões das bibliotecas usadas no capítulo (PyTorch, tiktoken, pandas, TensorFlow etc.).

```
In [180]: from importlib.metadata import version

pkgs = ["matplotlib",
        "numpy",
        "tiktoken",
        "torch",
        "tensorflow", # For OpenAI's pretrained weights
        "pandas"     # Dataset loading
        ]
for p in pkgs:
    print(f"{p} version: {version(p)}")
```

matplotlib version: 3.8.4  
numpy version: 1.26.4  
tiktoken version: 0.7.0  
torch version: 2.3.1  
tensorflow version: 2.8.0  
pandas version: 2.2.2



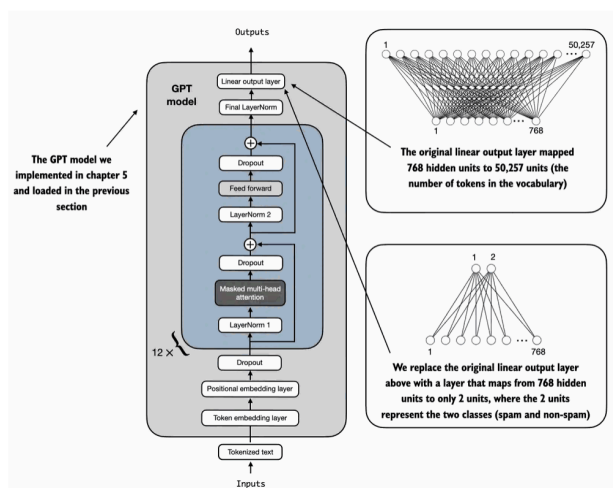
📺 **Na tela:** Título do capítulo 6 e o mapa das etapas com o passo 8 (Finetuning → Classifier) destacado.

🗣️ **Narração:** Capítulo 6. No mapa, estamos no passo 8, o ajuste fino para classificação. Partimos do modelo pré-treinado e o especializamos numa tarefa: dizer se uma mensagem de celular é spam ou não.

## Cena 25 — Nova camada de saída

07:23 → 07:41 · 18 segundos

### 6.5 Adicionando uma classification head



📺 **Na tela:** Diagrama: o modelo GPT à esquerda; no alto, à direita, a camada de saída original (768 → 50.257); embaixo, a nova camada (768 → 2).

🗣️ **Narração:** O diagrama mostra a única mudança. À esquerda, o GPT que já conhecemos. À direita, no alto, a camada de saída original, que ligava 768 números a

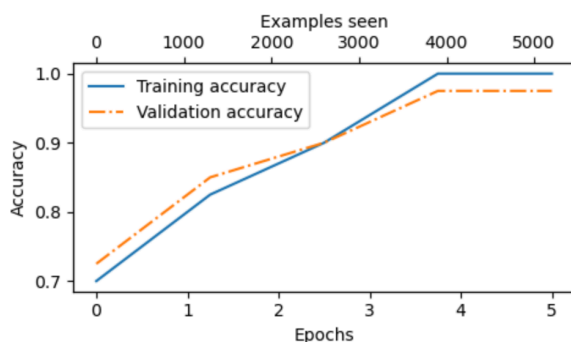
50.257 saídas, uma por palavra. Embaixo, a nova camada, com só duas saídas: spam e não spam.

## Cena 26 — Treino e acurácia

07:41 → 08:00 · 19 segundos

🇧🇷 **O que esta célula faz:** Plota a acurácia de treino e de validação ao longo das épocas, que sobe rapidamente e se aproxima de 100%.

```
In [218... epochs_tensor = torch.linspace(0, num_epochs, len(train_accs))
examples_seen_tensor = torch.linspace(0, examples_seen, len(train_accs))
plot_values(epochs_tensor, examples_seen_tensor, train_accs, val_accs, label="accuracy")
```



- Com base no gráfico de accuracy acima, vemos que o modelo atinge uma accuracy de treinamento e de validação relativamente alta após as épocas 4 e 5
- No entanto, precisamos lembrar que especificamos `eval_iter=5` na função de treinamento, o que significa que apenas estimamos o desempenho nos conjuntos de treinamento e de validação
- Podemos calcular o desempenho nos conjuntos de treinamento, validação e teste sobre o dataset completo, como mostrado abaixo

🇧🇷 **O que esta célula faz:** Mede a acurácia final nos conjuntos completos: cerca de 97% em treino e validação e 95,67% no teste.

```
In [219... train_accuracy = calc_accuracy_loader(train_loader, model, device)
val_accuracy = calc_accuracy_loader(val_loader, model, device)
test_accuracy = calc_accuracy_loader(test_loader, model, device)

print(f"Training accuracy: {train_accuracy*100:.2f}%")
print(f"Validation accuracy: {val_accuracy*100:.2f}%")
print(f"Test accuracy: {test_accuracy*100:.2f}%")
```

```
Training accuracy: 97.21%
Validation accuracy: 97.32%
Test accuracy: 95.67%
```

 **Na tela:** Gráfico da acurácia subindo ao longo de 5 épocas. Abaixo, a célula com a acurácia final: 97,21% (treino), 97,32% (validação) e 95,67% (teste).

 **Narração:** O gráfico mostra a acurácia, a porcentagem de acertos, subindo ao longo de cinco épocas até perto de 100 por cento. A última célula mede o resultado final: 97 por cento no treino e na validação e 95,67 no teste, com mensagens que o modelo nunca viu.

## Cena 27 — Classificando mensagens

08:00 → 08:14 · 14 segundos

- Vamos testá-la com alguns exemplos abaixo

 **O que esta célula faz:** Classifica uma mensagem prometendo prêmio em dinheiro: o modelo responde corretamente "spam".

```
In [221... text_1 = (  
    "You are a winner you have been specially"  
    " selected to receive $1000 cash or a $2000 award."  
)  
  
print(classify_review(  
    text_1, model, tokenizer, device, max_length=train_dataset.max_length  
))  
  
spam
```

 **O que esta célula faz:** Classifica um convite comum para jantar: o modelo responde corretamente "not spam".

```
In [222... text_2 = (  
    "Hey, just wanted to check if we're still on"  
    " for dinner tonight? Let me know!"  
)  
  
print(classify_review(  
    text_2, model, tokenizer, device, max_length=train_dataset.max_length  
))  
  
not spam
```

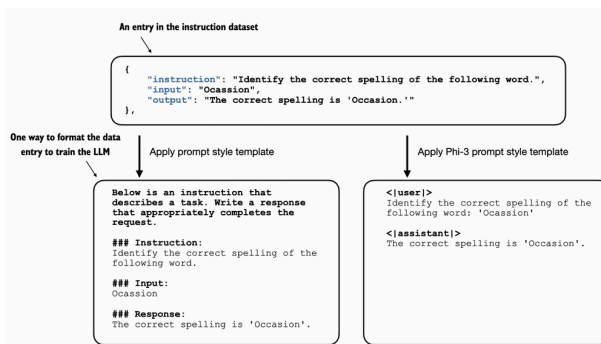
 **Na tela:** Duas células: uma mensagem oferecendo prêmio em dinheiro (resposta: spam) e um convite para jantar (resposta: not spam).

 **Narração:** Agora é só usar. A primeira mensagem diz que você ganhou mil dólares em dinheiro, e o modelo responde spam. A segunda é um convite para jantar, e o modelo responde not spam, uma mensagem normal.

## Capítulo 7 — Fine-tuning com instruções

### Cena 28 — Capítulo 7: instruções

08:14 → 08:32 · 18 segundos



- Neste capítulo, usamos a formatação de prompt no estilo Alpaca, que foi o template (modelo) de prompt original para instruction finetuning
- Abaixo, formatamos a entrada que passaremos ao LLM

 **O que esta célula faz:** Define a função que formata uma entrada no estilo de prompt do Alpaca, com a instrução e, se houver, o campo de entrada.

```
In [230]... def format_input(entry):  
    instruction_text = (  
        f"Below is an instruction that describes a task. "  
        f"Write a response that appropriately completes the request."  
        f"\n\n### Instruction:\n{entry['instruction']}"  
    )  
  
    input_text = f"\n\n### Input:\n{entry['input']}" if entry["input"] else ""  
  
    return instruction_text + input_text
```

- Uma resposta formatada com o campo de entrada (input) fica como mostrado abaixo

 **O que esta célula faz:** Monta um exemplo completo no formato Alpaca, com instrução, entrada e a resposta desejada.

```
In [231]... model_input = format_input(data[50])  
desired_response = f"\n\n### Response:\n{data[50]['output']}"  
  
print(model_input + desired_response)
```

Below is an instruction that describes a task. Write a response that appropriately completes the request.

### Instruction:  
Identify the correct spelling of the following word.

### Input:  
Occasion

### Response:  
The correct spelling is 'Occasion.'

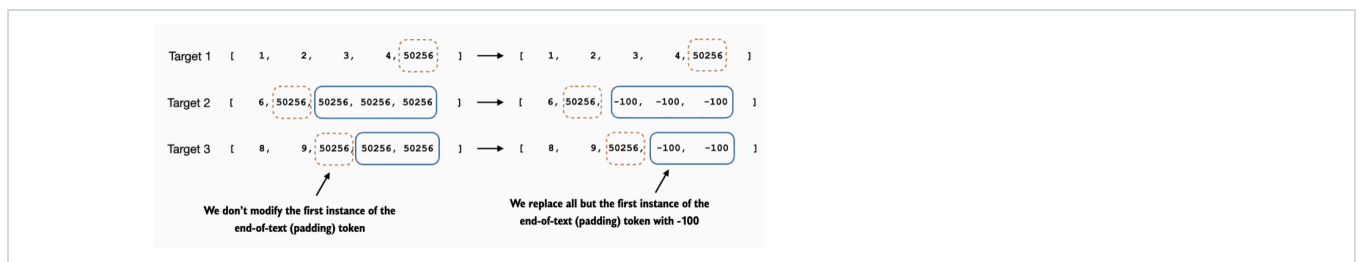
**Na tela:** Diagrama do formato Alpaca (instrução, entrada, resposta). Abaixo, a função que formata e um exemplo: instrução de grafia, entrada "Ocassion", resposta "Occasion".

**Narração:** Capítulo 7: ensinar o modelo a seguir instruções. O diagrama mostra o formato Alpaca: a instrução, uma entrada opcional e a resposta esperada. Abaixo, um exemplo real: a instrução pede a grafia correta de uma palavra, a entrada é Ocassion e a resposta, Occasion.

## Cena 29 — Montando os lotes

08:32 → 08:51 · 19 segundos · 2 telas em sequência, dividindo o tempo entre elas

### Tela A



### Tela B

**O que esta célula faz:** Nos alvos, os tokens de padding após o primeiro agora aparecem como -100, que será ignorado pela loss.

```
In [242...] inputs, targets = custom_collate_fn(batch)
print(inputs)
print(targets)

tensor([[ 0, 1, 2, 3, 4],
        [ 5, 6, 50256, 50256, 50256],
        [ 7, 8, 9, 50256, 50256]])
tensor([[ 1, 2, 3, 4, 50256],
        [ 6, 50256, -100, -100, -100],
        [ 8, 9, 50256, -100, -100]])
```

**Na tela:** Tela A: diagrama dos alvos; o primeiro 50256 é mantido e os demais viram -100. Tela B: a saída com entradas e alvos prontos.

**Narração:** Para treinar em lotes, completamos os textos curtos com o token de fim de texto, o 50256. O diagrama mostra um detalhe: nos alvos, mantemos o primeiro 50256 e trocamos os demais por menos 100, valor que a loss ignora. A saída mostra entradas e alvos prontos.

## Cena 30 — O modelo antes do ajuste

08:51 → 09:07 · 16 segundos · 2 telas em sequência, dividindo o tempo entre elas

### Tela A

- No entanto, em vez de carregar o menor modelo, de 124 milhões de parâmetros, carregamos a versão média (medium), com 355 milhões de parâmetros, já que o modelo de 124 milhões é pequeno demais para obter resultados qualitativamente razoáveis com instruction finetuning

## Tela B

 **O que esta célula faz:** Isola a resposta gerada: o modelo ainda não segue a instrução e apenas repete a frase e a instrução.

```
In [256... response_text = generated_text[len(input_text):].strip()
print(response_text)
```

### Response:

The chef cooks the meal every day.

### Instruction:

Convert the active sentence to passive: 'The chef cooks the

- Como podemos ver, o modelo ainda não é capaz de seguir as instruções; ele cria uma seção "Response", mas simplesmente repete a frase de entrada original e a instrução

 **Na tela:** Tela A: texto sobre o uso do GPT-2 medium (355 milhões de parâmetros). Tela B: a resposta do modelo antes do ajuste, que apenas repete a frase e a instrução.

 **Narração:** Aqui usamos o GPT-2 medium, com 355 milhões de parâmetros. Antes do ajuste fino, pedimos para passar uma frase para a voz passiva, e o modelo apenas repete a frase e a instrução. Ele ainda não sabe seguir instruções.

## Cena 31 — Treino e respostas

09:07 → 09:25 · 18 segundos · 2 telas em sequência, dividindo o tempo entre elas

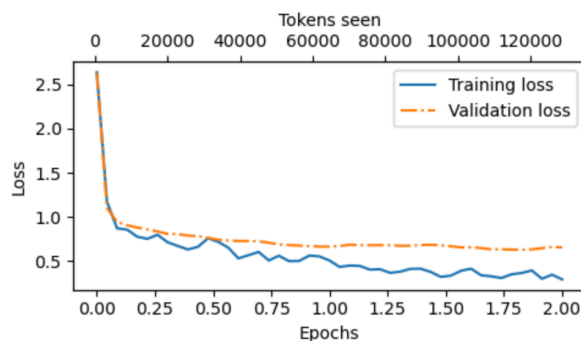
## Tela A

 **O que esta célula faz:** Plota as curvas de loss de treino e validação: queda brusca no início e leve overfitting a partir de cerca de 1 época.

```
In [260... import matplotlib.pyplot as plt
from suporte.previous_chapters_cap07 import plot_losses

plt.figure(figsize=(12, 6))
epochs_tensor = torch.linspace(0, num_epochs, len(train_losses))
plot_losses(epochs_tensor, tokens_seen, train_losses, val_losses)
```

<Figure size 1200x600 with 0 Axes>



- Como podemos ver, a loss cai bruscamente no início da primeira época, o que significa que o modelo começa a aprender rapidamente
- Podemos ver que um leve overfitting (sobreajuste) começa por volta de 1 época de treinamento

## Tela B

 **O que esta célula faz:** Gera respostas para três instruções do conjunto de teste e as compara com as respostas corretas; o modelo acerta bem as tarefas.

```
In [261... torch.manual_seed(123)

for entry in test_data[:3]:

    input_text = format_input(entry)

    token_ids = generate(
        model=model,
        idx=text_to_token_ids(input_text, tokenizer).to(device),
        max_new_tokens=256,
        context_size=BASE_CONFIG["context_length"],
        eos_id=50256
    )
    generated_text = token_ids_to_text(token_ids, tokenizer)
    response_text = generated_text[len(input_text):].replace("### Response:", "").strip()

    print(input_text)
    print(f"\nCorrect response:\n>> {entry['output']}")
    print(f"\nModel response:\n>> {response_text.strip()}")
    print("-----")
```

Below is an instruction that describes a task. Write a response that appropriately completes the request.

### Instruction:  
Rewrite the sentence using a simile.

### Input:  
The car is very fast.

Correct response:  
>> The car is as fast as lightning.

Model response:  
>> The car is as fast as a cheetah.  
-----

Below is an instruction that describes a task. Write a response that appropriately completes the request.

### Instruction:  
What type of cloud is typically associated with thunderstorms?

Correct response:  
>> The type of cloud typically associated with thunderstorms is cumulonimbus.

Model response:  
>> The type of cloud associated with thunderstorms is a cumulus cloud.  
-----

Below is an instruction that describes a task. Write a response that appropriately completes the request.

### Instruction:  
Name the author of 'Pride and Prejudice'.

Correct response:  
>> Jane Austen.

Model response:  
>> The author of 'Pride and Prejudice' is Jane Austen.  
-----

 **Na tela:** Tela A: gráfico das losses do ajuste fino. Tela B: três instruções do teste com a resposta correta e a resposta do modelo.

 **Narração:** O gráfico mostra a loss caindo bruscamente no início e terminando perto de 0,65 na validação. Nas respostas, o modelo cria uma comparação, as fast as a cheetah;

responde cumulus onde o certo era cumulonimbus, um erro próximo; e acerta a autora de Orgulho e Preconceito.

## Cena 32 — O Llama 3 como avaliador

09:25 → 09:44 · 19 segundos

- Como podemos ver, o modelo Llama 3 fornece uma avaliação razoável e também dá pontuação parcial quando uma resposta não está totalmente correta, como vemos no caso da resposta "cumulus cloud"
- Observe que o prompt anterior retorna avaliações muito prolixas; podemos ajustar o prompt para gerar respostas com números inteiros entre 0 e 100 (em que 100 é o melhor) e assim calcular uma nota média para o nosso modelo
- A avaliação das 110 entradas do conjunto de teste leva cerca de 1 minuto em um notebook M3 MacBook Air

 **O que esta célula faz:** Opcional: requer Ollama rodando localmente; não funciona no Colab. Pede ao Llama 3 apenas a nota numérica de cada uma das 110 respostas de teste; a nota média do modelo é 51,28.

```
In [269... def generate_model_scores(json_data, json_key):
    scores = []
    for entry in tqdm(json_data, desc="Scoring entries"):
        prompt = (
            f"Given the input `{format_input(entry)}` "
            f"and correct output `{entry['output']}`, "
            f"score the model response `{entry[json_key]}` "
            f"on a scale from 0 to 100, where 100 is the best score. "
            f"Respond with the integer number only."
        )
        score = query_model(prompt)
        try:
            scores.append(int(score))
        except ValueError:
            print(f"Could not convert score: {score}")
            continue

    return scores

scores = generate_model_scores(test_data, "model_response")
print(f"Number of scores: {len(scores)} of {len(test_data)}")
print(f"Average score: {sum(scores)/len(scores):.2f}\n")
```

```
Scoring entries: 100%|██████████| 110/110 [01:14<00:00, 1.47it/s]
Number of scores: 110 of 110
Average score: 51.28
```

- Nosso modelo atinge uma nota média acima de 50, que podemos usar como ponto de referência para comparar o modelo com outros modelos ou para testar outras configurações de treinamento que possam melhorá-lo
- Observe que o ollama não é totalmente determinístico (no momento em que escrevo), então os números que você obtiver podem diferir ligeiramente dos mostrados acima

 **Na tela:** Célula que pede ao Llama 3 uma nota de 0 a 100 para cada resposta; saída: 110 notas e média 51,28.

 **Narração:** Para avaliar respostas livres, usamos um modelo maior, o Llama 3, como juiz: ele compara cada resposta com a correta e dá uma nota de 0 a 100. Nas 110

respostas, o nosso modelo tirou média 51,28. Esta etapa exige o Ollama local e não roda no Colab.

## Encerramento

### Cena 33 — Encerramento

09:44 → 10:00 · 16 segundos

#### Encerramento

Ao longo deste notebook construímos um LLM completo, passo a passo:

1. transformamos texto em tokens e embeddings;
2. implementamos o mecanismo de atenção;
3. montamos a arquitetura GPT;
4. pré-treinamos o modelo e carregamos os pesos oficiais do GPT-2;
5. ajustamos o modelo para classificar spam;
6. ajustamos o modelo para seguir instruções e o avaliamos com outro LLM.

Esse é o mesmo ciclo usado, em escala muito maior, para criar modelos como o ChatGPT.



**Na tela:** A célula de encerramento, com a lista das seis etapas percorridas.



**Narração:** Assim fechamos o ciclo completo, resumido na tela: tokens, atenção, arquitetura, pré-treino e dois ajustes finos. É o mesmo caminho dos grandes modelos de linguagem, em escala muito maior. O notebook está pronto para você executar no Google Colab.